Chapter 4. AI Agent 노드 제대로 이해하기

> ⏱️ **배정 시간 55분** — Agent 구조 10분 · Memory 10분 · 실습 4-1 10분 · 실습 4-2 10분 · 실습 4-3 15분

📋 학습 목표

이 챕터를 마치면 다음을 할 수 있습니다.

- [ ] AI Agent 노드가 Basic LLM Chain과 무엇이 다른지 설명할 수 있다
- [ ] Model, Memory, Tools 세 구성요소의 역할을 구분할 수 있다
- [ ] Simple Memory를 연결해 대화 맥락을 유지시킬 수 있다
- [ ] Memory의 한계(기억 개수, 세션 구분)를 직접 확인하고 설명할 수 있다
- [ ] Calculator Tool로 견적 계산을, Code Tool로 영업일 계산을 Agent에 위임할 수 있다
- [ ] System Message로 에이전트의 응답 방식을 조정할 수 있다

📖 이 장의 용어

용어	영문 · 약어 풀이	설명
AI Agent 노드	—	질문을 받아 필요한 도구를 스스로 골라 호출하고 결과를 보고 다시 판단하는 노드. 1일차의 `Agent(...)`
System Message	시스템 메시지	Agent의 역할과 규칙을 적는 설정. 1일차의 Instruction, 일반적으로는 System Prompt
Memory	메모리, 대화 기억	이전 대화를 매 요청에 함께 실어 보내 맥락을 유지하는 서브 노드. Simple Memory는 최근 N개만 기억
Context Window Length	문맥 창 길이	Simple Memory가 기억할 최근 메시지 개수
Session / sessionId	세션 / 세션 식별자	대화를 묶는 단위와 그 고유 번호. 채팅창마다 다른 sessionId가 부여되어 대화가 섞이지 않음
Tool	도구	Agent가 답변 도중 호출할 수 있는 기능. Calculator, Code Tool, 서브 워크플로우 등

용어	영문 · 약어 풀이	설명
Calculator	계산기 Tool	수식을 받아 정확히 계산해 돌려주는 내장 Tool
Code Tool	—	JavaScript 코드를 Tool로 만드는 노드. 입력은 `query` 변수로 들어오고, `return`한 문자열이 결과
Description	설명	Tool이 무엇을 하고 어떤 입력을 받는지 적는 란. Agent는 이것만 보고 호출을 판단. 1일차의 Docstring
Logs 탭	로그, 기록	Agent가 어떤 Tool을 어떤 입력으로 호출했고 무엇을 받았는지 순서대로 보여주는 화면. 1일차의 Events 탭
공급가액 / 부가세	Supply Value / VAT(Value Added Tax)	공급가액은 세금 전 금액, 부가세는 그 10%. 합계 = 공급가액 × 1.1
영업일	Business Day	주말·공휴일을 제외한 근무일. 이 장의 Code Tool은 주말만 제외(공휴일은 Chapter 5)
튜닝	Tuning, 조정	System Message나 설정을 조금씩 바꿔 원하는 동작에 가까워지게 하는 작업

4.1 AI Agent 노드 구조

4.1.1 Basic LLM Chain과 무엇이 다른가

Chapter 3의 Basic LLM Chain은 "프롬프트 하나 → 응답 하나"로 끝났습니다. 중간에 뭔가를 조회하거나, 앞선 대화를 참고하거나, 답을 내기 전에 계산을 해보는 일은 하지 못합니다.

AI Agent 노드는 여기에 **판단과 반복**이 더해집니다. 질문을 받으면 "이걸 답하려면 무엇이 필요한가"를 스스로 따져보고, 필요하면 도구를 호출하고, 그 결과를 보고 다시 판단한 뒤 최종 답변을 만듭니다.

> 🔗 **1일차 연결**: 1일차 Chapter 3에서 Python 함수를 `tools=[...]`에 등록하면 LLM이 Docstring을 읽고 호출 여부를 판단했습니다. n8n의 AI Agent 노드는 그 구조를 그대로 노드로 옮긴 것입니다. 아래 표의 대응 관계를 기억해 두면 두 도구를 오가기 쉽습니다.

n8n	1일차 Google ADK
AI Agent 노드	`Agent(...)`
System Message	`instruction`
Chat Model	`model="gemini-..."`

n8n	1일차 Google ADK
Simple Memory + sessionId	`InMemorySessionService` • Session
Tool (Calculator, Code Tool 등)	`tools=[...]`에 등록한 함수
Tool의 Description	함수의 Docstring
Logs 탭	adk web의 Events 탭

연결 구조

Agent 노드도 혼자서는 동작하지 않습니다. Chapter 3에서 본 것과 같은 방식으로 아래쪽에 노드를 붙입니다. 다만 커넥터가 세 개로 늘어납니다.

```
AI Agent
   |
   ├── (Chat Model 커넥터) ── Google Gemini Chat Model     ← 필수. 생각하는 두뇌
   ├── (Memory 커넥터) ──────── Simple Memory 등            ← 선택. 대화 기억
   └── (Tool 커넥터) ──────────── Calculator, Code Tool 등   ← 선택. 여러 개 연결 가능
```

구성요소	필수 여부	없으면 어떻게 되나
Model	필수	노드가 아예 실행되지 않음
Memory	선택	매 메시지마다 처음 만난 사람처럼 응답
Tools	선택	도구 없이 아는 지식으로만 답변

4.1.2 Memory — 대화를 기억한다는 것

Memory를 연결하지 않으면 에이전트는 매 메시지를 독립된 요청으로 취급합니다.

Memory 없음	Memory 있음
"제 이름은 홍길동이에요" → "네, 반갑습니다"	동일
"제 이름이 뭐라고 했죠?" → **"모르겠습니다"**	**"홍길동이라고 하셨습니다"**

기술적으로는 Memory가 이전 대화 내용을 매 요청마다 함께 보내주는 방식으로 동작합니다. 모델이 "기억"하는 것이 아니라 **n8n이 매번 대화 기록을 첨부해주는 것**에 가깝습니다. 1일차 Chapter 5에서 "AI가 기억하는 것이 아니라 Session이 Context를 제공한다"고 배운 것과 같은 원리입니다.

4.1.3 Tools — 에이전트가 쓸 수 있는 능력

Tools는 에이전트가 답변을 만드는 도중에 호출할 수 있는 기능입니다. 무엇을 언제 쓸지는 에이전트가 스스로 판단합니다.

상황	필요한 Tool
정확한 계산이 필요한 질문 (견적, 부가세)	Calculator
날짜 계산, 문자열 가공 등 커스텀 로직	Code Tool
외부 API 조회	HTTP Request Tool
워크플로우로 만든 작업 위임	Call n8n Workflow Tool (Chapter 5)

Tool은 여러 개를 동시에 연결할 수 있고, 에이전트는 매 질문마다 "이 도구가 필요한가"를 판단합니다.

✅ 체크포인트 질문

- AI Agent와 Basic LLM Chain의 가장 큰 차이는? → **도구를 스스로 선택하고 반복 추론할 수 있다는 점**
- "이전에 말한 내 이름을 기억해줘"를 만족시키려면? → **Memory**
- n8n의 System Message는 1일차의 무엇에 해당하나요? → **Instruction**

4.2 Memory 설정

4.2.1 Simple Memory

가장 기본적인 Memory 노드입니다. **최근 N개의 메시지만 기억**하는 방식으로, 별도 데이터베이스 설정 없이 바로 쓸 수 있습니다.

설정 절차

1. AI Agent 노드의 `Memory` 커넥터 `+` 클릭
2. `Simple Memory` 검색 후 추가
3. `Context Window Length`(기억할 메시지 개수)를 `10`으로 설정

설정값	특징
5 이하	최근 대화만 기억, 토큰 절약
10 ~ 20	**일반적인 챗봇에 적합**
30 이상	긴 맥락 유지 가능하나 매 요청 토큰이 늘어남

> 💡 Simple Memory는 n8n이 켜져 있는 동안에만 기억을 유지합니다. n8n을 재시작하면 초기화됩니다. 실습용으로는 충분하며, 실제 서비스에서 대화 기록을 영구 보관하려면 별도 저장소 노드가 필요합니다.

4.2.2 세션(Session) 개념

Memory는 **세션 단위**로 대화를 구분합니다. Chat Trigger를 쓰면 대화창마다 고유한 `sessionId`가 자동 부여되어, 서로 다른 사용자의 대화가 섞이지 않습니다.

같은 채팅창에서 이어가야 이전 메시지를 기억하고, **새 채팅을 시작하면 새로운 세션**으로 취급되어 이전 기록을 참조하지 않습니다. 1일차 adk web에서 "+ New Session"을 눌렀을 때와 같은 동작입니다.

✅ 체크포인트 질문

- `Context Window Length`를 낮게 잡으면? → **토큰은 절약되지만 오래된 대화를 잊음**
- 채팅창을 새로 열면 이전 대화를 기억하지 못하는 이유는? → **sessionId가 달라져 별도 대화로 취급되기 때문**

4.3 Tool 연결 기초

4.3.1 Calculator

AI 모델은 큰 자릿수의 곱셈이나 복잡한 계산에서 의외로 자주 틀립니다. 언어를 다루도록 만들어진 모델이지 계산기가 아니기 때문입니다. 견적서처럼 **숫자가 틀리면 안 되는 업무**에서는 Calculator를 연결해 **실제 계산은 도구가 하고, AI는 결과를 해석해서 답변**하게 합니다.

설정 절차

1. AI Agent 노드의 `Tool` 커넥터 `+` 클릭
2. `Calculator` 검색 후 추가 — 별도 설정 없이 바로 사용 가능합니다

4.3.2 Code Tool

JavaScript 코드를 직접 작성해서 도구로 만드는 방식입니다. 정해진 노드로 처리하기 어려운 로직(날짜 계산, 문자열 가공 등)에 씁니다. 1일차에 Python 함수로 Tool을 만든 것과 같은 일을 JavaScript로 하는 것입니다.

설정 절차

1. `Tool` 커넥터 `+` → `Code Tool` 검색 후 추가 (버전에 따라 `Custom Code Tool`로 표시될 수 있습니다)
2. 노드 이름을 영문으로 바꾸고 `Description` 작성 (현재 버전은 별도의 Name 필드가 없고 **노드 이름이 곧 Tool 이름**입니다)
3. `Code` 영역에 로직 작성

동작 방식

- 에이전트가 이 도구를 호출하면서 전달한 입력값은 코드 안에서 `query` 변수로 들어옵니다 (문자열)
- 코드가 `return`한 문자열이 도구의 결과로 에이전트에게 돌아갑니다

> ⚠️ **Description이 이 노드의 핵심입니다.**
> 에이전트는 코드를 읽지 않습니다. **Description만 보고** 이 도구를 언제 호출할지, `query`에 무엇을 넣을지 판단합니다. "무엇을 위한 도구인지"와 "어떤 입력을 받는지"를 반드시 구체적으로 적으세요. 1일차의 Docstring과 역할이 같습니다.

✅ 체크포인트 질문

- 에이전트가 견적 금액 계산에서 틀리지 않게 하려면? → **Calculator 연결**
- Code Tool의 Description을 명확히 써야 하는 이유는? → **에이전트가 그 설명만 보고 호출 여부와 입력값을 판단하기 때문**

4.4 실습 — 사내 업무 비서

> 🎯 **목표**: Chat Trigger + AI Agent + Memory + Tool 2개를 연결해, 대화를 기억하면서 견적 계산과 영업일 계산을 정확히 해 주는 업무 비서를 완성한다.
> ⏱️ **소요 시간 35분** — 실습 4-1(Agent + Memory) 10분 → 4-2(Calculator) 10분 → 4-3(Code Tool) 15분

이 실습이 해결하는 업무

"단가 48,500원짜리 120개 견적 내면 부가세 포함 얼마지?", "11월 28일 납기까지 영업일로 며칠 남았지?" — 계산기와 달력을 번갈아 켜던 일을 채팅 한 줄로 처리합니다.

완성 구조

```
Chat Trigger → AI Agent ─┬─ Google Gemini Chat Model
                         ├─ Simple Memory
                         ├─ Calculator
                         └─ Code Tool (business_days_until)
```

4.4.1 실습 4-1. Agent 기본 연결과 Memory

1. 새 워크플로우 생성, 이름을 `실습4_업무비서`로 설정
2. `Chat Trigger` 노드 추가 (검색 시 `On chat message`로 표시될 수 있습니다)
3. 뒤에 `AI Agent` 노드 추가

4. `Chat Model` 커넥터에 `Google Gemini Chat Model` 연결 → Chapter 3의 Credential과 Flash 계열 모델 선택
5. AI Agent 노드의 `Add Option` → **System Message** 선택 후 아래 내용 입력
6. `Memory` 커넥터에 `Simple Memory` 연결, `Context Window Length`를 `10`으로 설정
7. 화면 하단 **Open chat** 버튼을 눌러 채팅창을 열고, 아래 두 문장을 순서대로 입력해 2번째에서 "지민"이라고 답하는지 확인

```
1번째: "안녕, 나는 총무팀 지민이야."
2번째: "내 이름이 뭐라고 했지?"
```

5번 단계에 입력할 System Message

```
당신은 총무팀의 업무를 돕는 사내 업무 비서입니다.
사용자의 질문에 친절하고 정확하게 답하세요.

규칙
- 금액 계산이 필요한 질문은 반드시 계산 도구를 사용해 정확한 값을 구하세요. 암산하지 마세요.
- 견적 금액을 물으면 공급가액과 부가세(10%), 합계를 각각 구분해서 답하세요.
- 금액은 원 단위 정수로 반올림하고 천 단위 쉼표를 붙여 표기하세요.
- 특정 날짜까지 남은 영업일을 물으면 반드시 business_days_until 도구를 사용하세요.
- 모르는 내용은 추측하지 말고 모른다고 답하세요.
- 답변은 3문장 이내로 간결하게 작성하세요.
```

> 💡 `business_days_until`은 실습 4-3에서 만들 도구의 이름입니다. 미리 적어 두어도 도구가 없을 때는 에이전트가 무시하거나 "도구가 없다"고 답합니다. 실습 4-3을 마치면 이 규칙이 동작하기 시작합니다.

🔬 Memory의 한계 확인해보기 (5분)

Memory가 무한정 기억하는 것이 아니라는 점을 직접 확인해봅니다.

1. `Context Window Length`를 `2`로 낮추고, 아래 네 문장을 순서대로 입력합니다
2. 4번째 질문에서 이름을 기억하지 못합니다. 창(window) 밖으로 밀려났기 때문입니다
3. 채팅창 우측 상단의 새 대화 시작 버튼을 눌러 새 세션을 열고, 다시 "내 이름이 뭐지?"라고 물어봅니다 → 역시 기억하지 못합니다. **세션이 달라졌기 때문**입니다
4. 확인 후 `Context Window Length`를 `10`으로 되돌립니다

```
1번째: "내 이름은 지민이야."
2번째: "나는 총무팀이야."
```

```
3번째: "이번 달 담당 업무는 사무용품 구매야."
4번째: "내 이름이 뭐라고 했지?"
```

이 두 가지가 실제 서비스에서 "왜 갑자기 기억을 못 하지?"의 대표적인 원인입니다.

4.4.2 실습 4-2. Calculator로 견적 계산

1. AI Agent의 `Tool` 커넥터에 `Calculator` 연결
2. 채팅창에 아래를 입력

```
"단가 48,500원짜리 모니터암 120개 견적 내줘. 부가세 포함 총액이 얼마야?"
```

3. AI Agent 노드를 클릭하고 우측 **Logs 탭**을 엽니다. Calculator가 실제로 호출되었는지, 어떤 수식(`48500 * 120`, `5820000 * 0.1` 등)이 전달되었는지 확인합니다
4. 답변에 공급가액 5,820,000원 / 부가세 582,000원 / 합계 6,402,000원이 정확히 나오는지 확인합니다

> 💡 Calculator는 `5820000 * 1.1`을 `6402000.000000001`처럼 끝자리에 오차가 있는 값으로 돌려줄 수 있습니다. 컴퓨터가 소수를 저장하는 방식에서 생기는 오차이며, System Message의 반올림 규칙이 최종 답변에서 이를 정리합니다. Logs 탭에서 이 값을 보면 오류가 아니라는 점을 안내합니다.

> 💡 **Logs 탭은 이 과정 내내 가장 중요한 디버깅 도구입니다.** 에이전트가 "왜 이렇게 답했는지"는 여기서만 알 수 있습니다. 도구를 호출했는지, 호출했다면 무엇을 넣었고 무엇을 받았는지가 순서대로 표시됩니다. 1일차 adk web의 Events 탭과 같은 역할입니다.

비교 실험: Calculator 연결선을 잠깐 끊고 같은 질문을 해 보세요. 모델이 암산한 값이 맞을 때도 있지만 틀릴 때도 있고, 틀려도 자신 있게 답합니다. 확인 후 다시 연결합니다.

4.4.3 실습 4-3. Code Tool로 영업일 계산

납기나 마감까지 "영업일로 며칠 남았는지"는 모델이 암산으로 맞히기 어려운 대표적인 계산입니다. 주말을 빼고 세어야 하기 때문입니다. 이 계산을 JavaScript 코드로 만들어 도구로 등록합니다.

1. AI Agent의 `Tool` 커넥터 `+` → `Code Tool` 추가
2. 노드 이름을 바꾸고 `Description` 입력 (노드를 더블클릭한 뒤 상단 이름 클릭)

항목	값
노드 이름	`business_days_until`

항목	값
Description	`오늘부터 특정 날짜까지 남은 영업일 수(주말 제외)를 계산합니다. 사용자가 납기, 마감, 특정 날짜까지 며칠 남았는지 물어볼 때 사용합니다. 입력값은 YYYY-MM-DD 형식의 날짜 하나입니다. 예: 2026-11-28`

> ⚠️ 노드 이름은 System Message에 적은 `business_days_until`과 글자까지 같아야 합니다. n8n은 노드 이름의 한글·공백 등 영문·숫자·밑줄(_)이 아닌 글자를 밑줄로 바꿔 Tool 이름을 만들기 때문에, 한글 이름을 붙이면 Agent가 이 도구를 이름으로 찾지 못합니다.

3. `Code` 영역에 아래 코드 입력

```
// 에이전트가 전달한 날짜 문자열 (예: "2026-11-28")
const input = String(query).trim();
const target = new Date(input);

if (isNaN(target.getTime())) {
  return '날짜 형식이 올바르지 않습니다. YYYY-MM-DD 형식으로 다시 전달하세요.';
}

// 오늘 날짜 (시간 부분 제거)
const today = new Date();
today.setHours(0, 0, 0, 0);
target.setHours(0, 0, 0, 0);

if (target <= today) {
  return input + '은(는) 오늘이거나 이미 지난 날짜입니다. 남은 영업일: 0일';
}

// 하루씩 이동하며 주말(토=6, 일=0)이 아닌 날만 셉니다
let count = 0;
const cursor = new Date(today);
while (cursor < target) {
  cursor.setDate(cursor.getDate() + 1);
  const day = cursor.getDay();
  if (day !== 0 && day !== 6) count++;
}
```

```
return input + '까지 남은 영업일: ' + count + '일 (주말 제외, 공휴일 미반영)';
```

부분	역할
`query`	에이전트가 넣어 준 입력값. Description에 "YYYY-MM-DD 하나"라고 적었으므로 그 형식으로 들어옵니다
`isNaN(...)` 검사	형식이 틀렸을 때 오류 대신 안내 문장을 반환. 1일차 Chapter 4의 "Tool 안에서 오류를 잡아 메시지로 반환"과 같은 원칙
`while` 반복	오늘 다음 날부터 목표일까지 하루씩 이동하며 평일만 셈
`return` 문자열	이 문장이 그대로 에이전트에게 전달됨

4. 채팅창에 아래를 입력하고 Logs 탭에서 `business_days_until`이 호출되었는지, `query`에 날짜가 `YYYY-MM-DD`로 들어갔는지 확인

```
"11월 28일 납기까지 영업일로 며칠 남았어?"
```

> 💡 사용자는 "11월 28일"이라고만 말했지만, 에이전트가 Description의 형식 안내를 읽고 `2026-11-28`로 바꿔서 전달합니다. 연도는 현재 날짜를 기준으로 추론합니다. 입력값 변환은 LLM이, 계산은 코드가 맡는 분업입니다.

5. 결과 문장에 "공휴일 미반영"이 붙어 있는 점을 확인합니다. 공휴일까지 반영하려면 외부 데이터가 필요한데, 이것이 Chapter 5에서 공휴일 조회 Tool을 만드는 이유입니다

4.4.4 종합 테스트

같은 채팅창에서 아래를 순서대로 입력합니다.

```
1. "안녕, 나는 총무팀 수현이야. 이번 주에 사무용품 발주 정리 중이야."
2. "A4 용지 한 박스 23,000원인데 35박스 발주하면 부가세 포함 얼마야?"
3. "그 발주 납기가 12월 10일인데 영업일로 며칠 남았어?"
4. "아까 내가 무슨 일 정리 중이라고 했지?"
```

정상 동작한다면, 2번은 **Calculator**가 계산하고(합계 885,500원), 3번은 **Code Tool**이 영업일을 세고, 4번에서 **Memory**가 1번 대화를 불러와 답변합니다. Logs 탭에서 각 질문마다 어떤 도구가 호출되었는지 확인합니다.

튜닝 아이디어

상황	조정 방법
답변이 너무 길다	System Message의 "3문장 이내"를 더 앞쪽에 배치
계산 도구를 안 쓰고 암산하려 한다	"암산하지 마세요"를 "반드시 계산 도구의 결과만 사용하세요"로 강화
날짜 도구를 안 부른다	Code Tool의 Description에 "납기", "마감", "D-day" 등 사용자가 쓸 법한 단어를 추가
모르는 것도 그럴듯하게 지어낸다	"확실하지 않으면 모른다고 답하세요" 추가 + Temperature 낮추기

🎉 완성 확인

이전 대화를 기억하고, 금액 질문에는 Calculator를, 날짜 질문에는 Code Tool을 호출해 정확한 값을 답하면 실습 완료입니다. **Save**합니다. 이 워크플로우는 Chapter 5에서 복제해 확장합니다.

📌 Chapter 4 요약

개념	핵심 내용
AI Agent vs Basic LLM Chain	Agent는 도구를 스스로 선택하고 반복 추론 가능
3대 구성요소	Model(필수) + Memory(선택) + Tools(선택)
Simple Memory	최근 N개 메시지만 기억. 창을 넘어가면 잊음
세션	채팅창마다 sessionId 부여. 새 채팅 = 새 세션 = 기억 초기화
Calculator	금액·수량 계산을 모델 대신 수행
Code Tool	`query`로 입력받아 문자열을 `return`. **Description이 호출 판단의 전부**
System Message	Add Option에서 설정. 에이전트의 역할과 규칙 정의 (1일차 Instruction)
Logs 탭	에이전트가 어떤 도구를 왜 호출했는지 확인하는 유일한 창구 (1일차 Events 탭)
Tool 이름	Code Tool·Call n8n Workflow Tool은 노드 이름이 Tool 이름. 영문·숫자·밑줄(_)만 사용
실습 4-1~4-3	Chat Trigger → AI Agent(Model+Memory+Calculator+Code Tool) 업무 비서 완성

다음 Chapter 예고: Chapter 5에서는 **워크플로우 자체를 도구로 만드는 법**을 배웁니다. 날씨·환율·공휴일 조회를 각각 Tool로 만들어, "다음 주 도쿄 출장 준비해 줘" 한 마디에 세 가

지를 한 번에 알려주는 출장 준비 비서를 완성합니다.